Yes — **there are real errors in this model**, and because of them Django will fail on **makemigrations / migrate / createsuperuser**.
I'll explain **what's wrong**, **why it happens**, and then give you the **correct production-ready fix**.

❌ Problems in Your Current Code

1️⃣ `AbstractUser` already has a `username` field

You inherited from `AbstractUser`, but then you changed:

```
USERNAME_FIELD = 'email'
REQUIRED_FIELDS = []
```

⚠️ **Problem:**
`AbstractUser` **still requires** `username`, but you never removed it.

So Django gets confused:

- Login expects `email`
- Model still has `username`
- Manager probably expects email
  ➡️ **Result:** errors during create/update users

2️⃣ `email` is NOT unique

```
email = models.EmailField()
```

⚠️ **Problem:**
If `email` is `USERNAME_FIELD`, it **MUST be unique**.

Otherwise Django raises:

> USERNAME_FIELD must be unique

3️⃣ `CustomUserManager` + `AbstractUser` mismatch

When you override `USERNAME_FIELD`, your **manager must handle email-based users**.

If your `CustomUserManager` is written for `AbstractBaseUser`, this mismatch causes runtime errors.

4️⃣ Missing `null=True / blank=True`

Without these, Django admin + forms often break during user creation.

✅ Correct & Safe Solution (Recommended)

✔ Option A (BEST PRACTICE):

Use **email login** and **remove username completely**

✅ Correct User Model

```
from django.db import models
from django.contrib.auth.models import AbstractUser
from users.managers import CustomUserManager


class User(AbstractUser):
    username = None  # 🔴 REMOVE username
    email = models.EmailField(unique=True)

    address = models.TextField(blank=True, null=True)
    phone_number = models.CharField(max_length=15, blank=True, null=True)

    USERNAME_FIELD = 'email'
    REQUIRED_FIELDS = []

    objects = CustomUserManager()

    def __str__(self):
        return self.email
```

✅ Required `CustomUserManager`

```
from django.contrib.auth.models import BaseUserManager


class CustomUserManager(BaseUserManager):
```

```
    def create_user(self, email, password=None, **extra_fields):
        if not email:
            raise ValueError("Email is required")

        email = self.normalize_email(email)
        user = self.model(email=email, **extra_fields)
        user.set_password(password)
        user.save(using=self._db)
        return user

    def create_superuser(self, email, password=None, **extra_fields):
        extra_fields.setdefault("is_staff", True)
        extra_fields.setdefault("is_superuser", True)
        extra_fields.setdefault("is_active", True)

        if not extra_fields.get("is_staff"):
            raise ValueError("Superuser must have is_staff=True")
        if not extra_fields.get("is_superuser"):
            raise ValueError("Superuser must have is_superuser=True")

        return self.create_user(email, password, **extra_fields)
```

⚠️ MUST DO (Very Important)

1️⃣ `settings.py`

```
AUTH_USER_MODEL = 'users.User'
```

⚠️ Add this **BEFORE first migration**

2️⃣ If migrations already exist ❌

Run:

```
rm -rf users/migrations
python manage.py makemigrations
python manage.py migrate
```

(If production → we handle differently)

✅ Final Checklist

Item	Status
Remove username	✅
Email unique	✅
Email login works	✅
Admin panel works	✅
Create/update user works	✅
JWT / Djoser compatible	✅

🔥 Pro Tip (Interview-Ready Answer)

> **Why not use AbstractUser with email login?**
> Because it keeps unnecessary username logic.
> For clean design → remove username or use AbstractBaseUser.

If you want:

- ✅ **Admin customization**
- ✅ **JWT / Djoser setup**
- ✅ **Role-based users (Admin, Vendor, Customer)**

Just tell me 👍

```
from django.contrib import admin
from django.contrib.auth.admin import UserAdmin
from users.models import User


@admin.register(User)
class CustomUserAdmin(UserAdmin):
    model = User

    list_display = ('email', 'first_name', 'last_name', 'phone_number',
'is_active')
    list_filter = ('is_staff', 'is_active', 'is_superuser')

    ordering = ('email',)
    search_fields = ('email', 'first_name', 'last_name')

    fieldsets = (
        (None, {'fields': ('email', 'password')}),
```

```
        ('Personal Info', {
            'fields': ('first_name', 'last_name', 'address',
'phone_number')
        }),
        ('Permissions', {
            'fields': ('is_active', 'is_staff', 'is_superuser', 'groups',
'user_permissions')
        }),
        ('Important dates', {'fields': ('last_login', 'date_joined')}),
    )

    add_fieldsets = (
        (None, {
            'classes': ('wide',),
            'fields': ('email', 'password1', 'password2', 'is_staff',
'is_superuser'),
        }),
    )

    readonly_fields = ('last_login', 'date_joined')
```

Perfect 👍
Your `admin.py` code is now **correct**. I'll explain **each part clearly**, line-by-line, so you fully understand **why it's written this way** and **what problem it solves**.

1️⃣ What this file does (Big Picture)

This file **customizes Django Admin** so that:

- Users log in with **email instead of username**
- Admin pages **don't crash**
- You can **add / edit users cleanly**
- Permissions, groups, and dates work properly

Because you **removed** **`username`**, the default `UserAdmin` must be overridden.

2️⃣ Imports

```
from django.contrib import admin
from django.contrib.auth.admin import UserAdmin
from users.models import User
```

Why?

- `UserAdmin` → Django's built-in admin logic (permissions, hashing, forms)
- We **extend** it instead of writing everything from scratch
- `User` → your custom user model

3️⃣ `@admin.register(User)`

```
@admin.register(User)
class CustomUserAdmin(UserAdmin):
```

Why?

This registers the `User` model in admin using **your custom admin class**.

Equivalent to:

```
admin.site.register(User, CustomUserAdmin)
```

But cleaner and preferred.

4️⃣ `model = User`

```
model = User
```

Tells Django:

> This admin config is for the `users.User` model

5️⃣ `list_display`

```
list_display = ('email', 'first_name', 'last_name', 'phone_number',
'is_active')
```

Effect in Admin List View:

You see columns:

| Email | First Name | Last Name | Phone | Active |

Without this, admin would show only `__str__()`.

6️⃣ `list_filter`

```
list_filter = ('is_staff', 'is_active', 'is_superuser')
```

Adds sidebar filters:

✔ Staff
✔ Active
✔ Superuser

Helps admins filter users easily.

7️⃣ Ordering & Search

```
ordering = ('email',)
search_fields = ('email', 'first_name', 'last_name')
```

Why?

- Users are sorted by email
- Admin search bar works properly
- Required because **username no longer exists**

8️⃣ `fieldsets` (EDIT USER PAGE)

```
fieldsets = (
    (None, {'fields': ('email', 'password')}),
```

Section 1

Basic authentication fields.

```
('Personal Info', {
    'fields': ('first_name', 'last_name', 'address', 'phone_number')
```

```
}),
```

Section 2

User profile data.

```
('Permissions', {
    'fields': ('is_active', 'is_staff', 'is_superuser', 'groups',
'user_permissions')
}),
```

Section 3 (VERY IMPORTANT)

Controls:

- Access to admin
- Group-based permissions
- Model-level permissions

⚠️ `user_permissions` must be plural (common bug)

```
('Important dates', {'fields': ('last_login', 'date_joined')}),
```

Section 4

Read-only system-managed fields.

9️⃣ `add_fieldsets` (ADD USER PAGE)

```
add_fieldsets = (
    (None, {
        'classes': ('wide',),
        'fields': ('email', 'password1', 'password2', 'is_staff',
'is_superuser'),
    }),
)
```

Why this is REQUIRED

When clicking **“Add User”**:

- Default Django expects `username`
- You removed it
- Without this → admin crashes

This defines:

- Email
- Password confirmation
- Staff / superuser flags

🔟 `readonly_fields`

```
readonly_fields = ('last_login', 'date_joined')
```

Why?

These fields are:

- Automatically managed by Django
- Should not be editable manually

Admin displays them but prevents changes.

✅ FINAL RESULT (What You Achieved)

✔ Email-based authentication
✔ Clean admin UI
✔ No username dependency
✔ Secure permission handling
✔ Production-ready admin setup

🧠 Interview-Level Summary

> When customizing Django's User model, you must also override `UserAdmin` to match the new authentication fields; otherwise admin forms will break.

If you want next:

- 🔐 JWT login with email
- 🧑‍💼 Roles: Admin / Seller / Buyer
- 🛒 E-commerce user permissions

Just say the word 👍
ভালো প্রশ্ন 👍
সংক্ষেপে বললে 👉 **না, সব model-এর জন্য admin customization করা বাধ্যতামূলক না।**
কিন্তু **কিছু ক্ষেত্রে অবশ্যই করতে হয়**, না করলে admin ভেঙে যায় বা খারাপ UX হয়।

আমি **clear rule + checklist + example** দিয়ে বুঝাচ্ছি।

🔹 1️⃣ Admin customization কি?

Django Admin এ:

- কোন field দেখাবে
- কোনটা edit করা যাবে
- কোনটা filter/search হবে
  এইসব control করাকেই **admin customization** বলে।

🔹 2️⃣ কখন admin customization MUST (বাধ্যতামূলক)?

✅ MUST যখন👇

🔴 1. Custom User Model ব্যবহার করলে (100% MUST)

যেমন তুমি করেছো:

```
class User(AbstractUser):
    username = None
    email = models.EmailField(unique=True)
```

👉 Default `UserAdmin` তখন আর কাজ করে না
👉 **admin.py override না করলে admin crash করবে**

✔ তাই User model এ admin customization **অবশ্যই লাগবে**

🔴 2. Username remove / auth change করলে

যদি:

- `USERNAME_FIELD` বদলাও
- `AbstractBaseUser` ব্যবহার করো

👉 Admin forms নিজে নিজে বুঝতে পারে না
👉 `add_fieldsets`, `fieldsets` দিতে হয়

🔹 3️⃣ কখন admin customization OPTIONAL?

✅ Optional যখন👇

🟢 1. Normal model (Product, Order, Blog, Category)

```
class Product(models.Model):
    name = models.CharField(max_length=100)
    price = models.FloatField()
```

এইটা **না করলেও চলে**:

```
admin.site.register(Product)
```

Admin ঠিকঠাক কাজ করবে 👍

🔹 4️⃣ কিন্তু কেন admin customize করা ভালো?

যদিও optional, **production project-এ strongly recommended**

Example without customization 😐

- সব field একসাথে
- search নাই
- filter নাই

Example with customization 😍

- Search
- Filter
- List columns
- Read-only fields

🔹 5️⃣ Minimum admin customization (BEST PRACTICE)

👉 **যেকোন model এর জন্য এইগুলো থাকলে ভালো**

```
@admin.register(Product)
class ProductAdmin(admin.ModelAdmin):
    list_display = ('name', 'price', 'created_at')
    search_fields = ('name',)
    list_filter = ('created_at',)
```

🔹 6️⃣ Admin customization checklist (MEMORIZE THIS)

Situation	Admin customization
Custom User model	🔴 MUST
AbstractBaseUser	🔴 MUST
USERNAME_FIELD change	🔴 MUST
Normal model	🟢 Optional
Production project	✅ Recommended

🔹 7️⃣ Interview-Ready Answer 🔥

> **Is admin customization mandatory in Django?**
> No, except for custom user models or authentication changes.
> For normal models it's optional but recommended for better usability.

🔹 8️⃣ তোমার case summary (VERY IMPORTANT)

তুমি যেহেতু:

- Custom User model
- Email login
- Username removed

👉 **Admin customization না করলে project চলতই না**
👉 তুমি সঠিক কাজ করেছো ✅

চল একদম স্টেপ-বাই-স্টেপ দেখি:

🛠️ Multi-role Assign & Access Control in Django (Step by Step)

1️⃣ Multi-role Assign করার জায়গা

সাধারণত ৩ জায়গায় আপনি role (group) assign করতে পারেন:

(A) User Registration / Signup Logic এ

- ইউজার রেজিস্ট্রেশন হলে আপনি তার জন্য ডিফল্ট role দিতে পারেন।
- যেমন, নতুন ইউজারকে `Customer` role assign করা।

```
from django.contrib.auth.models import Group

def create_user_with_role(email, password):
    user = User.objects.create_user(email=email, password=password)
    customer_group = Group.objects.get(name='Customer')
    user.groups.add(customer_group)  # role assign here
    user.save()
    return user
```

(B) Admin Panel থেকে

- Django admin panel এ যান।
- User এডিট পেজে গিয়ে **Groups** সেকশন থেকে এক বা একাধিক role (group) select করে save করুন।

(C) API বা Custom View থেকে

- কোড থেকে যেকোন সময়:

```
seller_group = Group.objects.get(name='Seller')
customer_group = Group.objects.get(name='Customer')
user.groups.add(seller_group, customer_group)
```

2️⃣ Multi-role Access Control কিভাবে দিবেন?

(A) Simple view function এ:

```
def my_view(request):
    if request.user.groups.filter(name='Seller').exists():
        # Seller specific logic
        pass
    if request.user.groups.filter(name='Customer').exists():
        # Customer specific logic
        pass
```

(B) Django REST Framework এ Permission Class Example:

```
from rest_framework.permissions import BasePermission

class IsSellerOrCustomer(BasePermission):
    def has_permission(self, request, view):
        return (
            request.user.is_authenticated and
            request.user.groups.filter(name__in=['Seller',
'Customer']).exists()
        )
```

(C) Template এ:

```
{% if 'Seller' in request.user.groups.all.values_list 'name' %}
    <!-- Seller specific menu -->
{% endif %}
```

3️⃣ কোথায় Roles (Groups) Create করবেন?

(A) Admin panel থেকে:

- Login to admin → Authentication and Authorization → Groups → Add groups like Seller, Customer, Admin

(B) Migration / Startup Script এ:

```
from django.contrib.auth.models import Group

roles = ['Admin', 'Seller', 'Customer']
for role in roles:
    Group.objects.get_or_create(name=role)
```

4️⃣ Best Practice Summary

Step	Location / Method
Create Roles	Admin panel / Migration / Startup
Assign Roles	Signup logic / Admin panel / API
Check Access	View logic / Permission classes

5️⃣ BONUS: Multi-role check - ইউজার একাধিক role পাবে, কিভাবে check করবেন?

```
if request.user.groups.filter(name__in=['Seller', 'Admin']).exists():
    # ইউজার Seller অথবা Admin role পেয়েছে
```

🎯 আপনার জন্য Next Steps?

- Multi-role ভিত্তিক API permission তৈরী
- Role অনুযায়ী dynamic UI
- Role-based admin customization

যদি চান, বলুন — আমি ডিটেইল দিয়ে সাহায্য করবো! 👍

Example explain

🛒 ই-কমার্সের জন্য মাল্টি-রোল সিস্টেম (Django Groups)

---

১. Roles কীভাবে define করবো? (Create groups)

A. Django Admin থেকে (সবচেয়ে সহজ)

- লগইন করো admin এ
- Authentication → Groups → Add Group → Add roles (Admin, Seller, Customer, Delivery, Support)

B. Startup এ Automatically Create করবো (Better)

`users/apps.py` ফাইলে (তোমার users অ্যাপের apps.py):

```
from django.apps import AppConfig

class UsersConfig(AppConfig):
    default_auto_field = 'django.db.models.BigAutoField'
    name = 'users'

    def ready(self):
        from django.contrib.auth.models import Group
        roles = ['Admin', 'Seller', 'Customer', 'Delivery', 'Support']
        for role in roles:
            Group.objects.get_or_create(name=role)
```

এভাবে সার্ভার চালু হলে রোলগুলো ডাটাবেজে তৈরি থাকবে।

---

২. User Model (Clean রাখবে, role field রাখবে না)

```
from django.contrib.auth.models import AbstractUser

class User(AbstractUser):
    email = models.EmailField(unique=True)
    phone_number = models.CharField(max_length=15, blank=True, null=True)

    USERNAME_FIELD = 'email'
    REQUIRED_FIELDS = []

    def __str__(self):
        return self.email
```

---

৩. রোল Assign করার ৩ জায়গা (যেখানে প্রয়োজন)

(A) User signup বা registration সময়:

```
from django.contrib.auth.models import Group

def create_user_with_role(email, password, role_name='Customer'):
    user = User.objects.create_user(email=email, password=password)
    group = Group.objects.get(name=role_name)
    user.groups.add(group)
    user.save()
    return user
```

(B) Admin থেকে ম্যানুয়ালি:

- User Edit → Groups → সিলেক্ট করে সেভ করো।

(C) API বা Custom ভিউ থেকে:

```
user.groups.add(Group.objects.get(name='Seller'))
user.groups.add(Group.objects.get(name='Delivery'))
```

একজন ইউজার একাধিক গ্রুপ পেতে পারে।

---

৪. রোল অনুযায়ী Access Control (Permission)

A. DRF Permission Class (Role-Based Access Control)

```
from rest_framework.permissions import BasePermission

class IsAdmin(BasePermission):
    def has_permission(self, request, view):
        return request.user.is_authenticated and
request.user.groups.filter(name='Admin').exists()

class IsSeller(BasePermission):
    def has_permission(self, request, view):
        return request.user.is_authenticated and
request.user.groups.filter(name='Seller').exists()

class IsDeliveryOrSupport(BasePermission):
    def has_permission(self, request, view):
        return request.user.is_authenticated and
request.user.groups.filter(name__in=['Delivery', 'Support']).exists()
```

B. View এ ব্যবহার:

```
from rest_framework.views import APIView

class ProductCreateView(APIView):
    permission_classes = [IsSeller]

    def post(self, request):
        # seller-only product create logic
        pass
```

C. একাধিক role দিয়ে permission দিতে চাইলে:

```
class IsSellerOrAdmin(BasePermission):
    def has_permission(self, request, view):
        return request.user.is_authenticated and
request.user.groups.filter(name__in=['Seller', 'Admin']).exists()
```

---

৫. Role চেক করার সাধারণ পদ্ধতি (View, Template, Function)

```
if request.user.groups.filter(name='Admin').exists():
    # Admin logic
elif request.user.groups.filter(name='Seller').exists():
    # Seller logic
```

---

৬. রোল বেজড UI ও Menu Control (Django Template)

```
{% if request.user.groups.filter(name='Admin').exists %}
    <a href="/admin-dashboard">Admin Dashboard</a>
{% endif %}

{% if request.user.groups.filter(name='Seller').exists %}
    <a href="/seller-products">Manage Products</a>
{% endif %}
```

---

৭. সার্বিক Flow Summary

Component	Location / Code
Role definition	Django Group (Admin or apps.py)
User model	Custom User (no role field)
Role assignment	Signup, Admin, API view
Permission check	DRF permission classes, view logic, template

---

৮. Interview Style Summary

> In production Django e-commerce projects, roles are best managed via Django Groups. Users can have multiple roles by being members of multiple groups. Permissions and access are controlled by checking group membership in views or API permissions.

তোমার জন্য Final Advice:

- User model কে clean রাখো
- Role গুলো Django Groups-এ রাখো
- User ↔ Group relationship ব্যবহার করো
- DRF বা View permissions-এ Group membership চেক করো
- একাধিক Role assign করো সহজেই `user.groups.add(...)` দিয়ে

চাও তাহলে আমি তোমার জন্য **full example Django REST Framework API** তৈরি করে দিতে পারি যা role-based access control করবে?
বা **Admin panel customization** যেখানে roles সহজে assign হবে?

বলেই দিও 👍

অবশ্যই! নিচে আমি তোমাকে `apps.py` **-তে Django Group (Role) auto-create করার পুরো example** দিচ্ছি, যা তোমার প্রজেক্টে রোলগুলো ডাটাবেজে একবার তৈরি করে দিবে।

Complete Example: apps.py এ Role (Group) Auto-Create করা

Step 1: users/apps.py ফাইল এডিট করো

```
from django.apps import AppConfig
from django.db.models.signals import post_migrate

class UsersConfig(AppConfig):
    default_auto_field = 'django.db.models.BigAutoField'
    name = 'users'

    def ready(self):
        from django.contrib.auth.models import Group
        from django.db.models.signals import post_migrate
        from django.dispatch import receiver

        @receiver(post_migrate)
        def create_user_roles(sender, **kwargs):
            if sender.name == self.name:
                roles = ['Admin', 'Seller', 'Customer', 'Delivery',
'Support']
                for role in roles:
                    Group.objects.get_or_create(name=role)
```

Step 2: settings.py এ ensure করো apps.py ঠিকমতো লোড হচ্ছে

`INSTALLED_APPS` -এ তোমার app যুক্ত আছে যেমন:

```
INSTALLED_APPS = [
    # ... other apps
    'users.apps.UsersConfig',  # এখানে apps.py ফাইলের সম্পূর্ণ পাথ দিন
    # 'users', # এটার পরিবর্তে apps.py path দিতে হবে
]
```

Step 3: Runserver বা migrate করলে Auto-create হবে

যখন তোমরা সার্ভার চালাবে:

```
python manage.py runserver
```

বা migrate করবে:

```
python manage.py migrate
```

`ready()` method একবার execute হবে এবং ডাটাবেজে রোলগুলো তৈরি হবে যদি না আগে থাকে।

কেন এটা ভালো?

- রোলগুলো একবারেই তৈরি হবে, তোমার ম্যানুয়াল হস্তক্ষেপ লাগবে না।
- ডাটাবেজে রোল মিসিং হওয়ার সমস্যা থাকবে না।
- Version control এ রাখা যাবে না, তবে dev/production environment সব সময় sync থাকবে।

Extra Tip: Avoid Multiple Inserts

`get_or_create()` ensures একাধিকবার একই নামের গ্রুপ তৈরি হবে না।

প্রয়োজনে আরো সাহায্য চাইলে বলো!

তোমার ই-কমার্স প্রজেক্টে এই রোলগুলো দিয়ে পরের ধাপগুলো যেমন role assign, permission control ইত্যাদি শেখাতে পারি। 😊
না, `INSTALLED_APPS`-এ **`users.apps.UsersConfig`** দিলে আর আলাদাভাবে `'users'` লিখার দরকার হয় না।

কারণ:
`users.apps.UsersConfig` হচ্ছে তোমার apps.py এর ক্লাসের ফুল পাথ, যা Django কে বলে এই অ্যাপের কনফিগারেশন কোনটা। Django তখন এই ক্লাস থেকে অ্যাপ লোড করবে।

সুতরাং, তোমার `INSTALLED_APPS` হবে এমন:

```
INSTALLED_APPS = [
    # অন্যান্য অ্যাপস
    'users.apps.UsersConfig',  # এই লাইনেই যথেষ্ট
    # 'users',  # এটা বাদ দাও
]
```

আরেকটু সহজে বললে:

- `'users.apps.UsersConfig'` লিখলে তোমার অ্যাপের apps.py ফাইলটা ব্যবহার হবে।
- `'users'` লিখলে Django ডিফল্ট apps.py ছাড়া সেই অ্যাপটাকে ধরে নিবে।

সামারি:

তোমার লিখা	দরকার?	অর্থ
`'users'`	না	সাধারন অ্যাপ
`'users.apps.UsersConfig'`	হ্যাঁ	apps.py এর Config ক্লাস ব্যবহার করে

যদি তোমার apps.py-তে কোনো **`ready()`** মেথড থাকে যেটা চালাতে চাও, তাহলে অবশ্যই `'users.apps.UsersConfig'` ব্যবহার করো।

কোনো সন্দেহ থাকলে বলো!
না, **দুটোই একসাথে দিতে হয় না** — তুমি বা তো `'users'` দিবে, বা `'users.apps.UsersConfig'` দিবে।

অর্থাৎ:

- যদি **`users/apps.py`** **তে** **`UsersConfig`** **ক্লাস আছে** আর তুমি সেটার `ready()` মেথড চালাতে চাও →
  তাহলে `INSTALLED_APPS` -এ দিবে `'users.apps.UsersConfig'`
- আর যদি apps.py এর কোনো কাস্টম ক্লাস না থাকে বা সেটা ব্যবহার না করতে চাও →
  তাহলে শুধু `'users'` দিলেই হবে।

সহজে:

```
# apps.py তে ready() ব্যবহার করলে:
INSTALLED_APPS = [
    # ...
    'users.apps.UsersConfig',
]

# apps.py তে কিছু না থাকলে বা ব্যবহার না করলে:
INSTALLED_APPS = [
```

```
    # ...
    'users',
]
```

তোমার ই-কমার্স প্রজেক্টে `ready()` মেথড ব্যবহার করছো তো তাই `'users.apps.UsersConfig'` দিবে।

Auto relation `thakbe ki` for user and group model

যদি তুমি `User` **মডেল** এ `groups` **field কাস্টমাইজ বা রিপ্লেস না করো**, অর্থাৎ

- তোমার ইউজার ক্লাস যদি `AbstractUser` বা `AbstractBaseUser` থেকে ইনহেরিট করে,
- আর তুমি সেখানে `groups` নামে কোনো নতুন ফিল্ড ডিক্লেয়ার না করো,

তাহলে Django এর ডিফল্ট **Many-to-Many relation between User and Group** থাকবে স্বয়ংক্রিয়ভাবে।

বিস্তারিত:

- Django এর `AbstractUser` বা `AbstractBaseUser` ক্লাসেই `groups = ManyToManyField(Group)` ফিল্ড থাকে।
- তাই তোমার কাস্টম ইউজার মডেলে সেটা না দিলে, ডিফল্ট ভাবে Django এই সম্পর্ক মেনেই কাজ করবে।
- অর্থাৎ, তুমি `user.groups.add()`, `user.groups.remove()`, `user.groups.all()` এগুলো ব্যবহার করতে পারবে কোন সমস্যা ছাড়াই।

কখন কাস্টমাইজ করতে হবে?

- সাধারণ ক্ষেত্রে দরকার হয় না।
- যদি রিলেশন বা ফিল্ডের কোনো behavior বদলাতে চাও (যেমন `related_name` পরিবর্তন, বা extra ফিল্ড যুক্ত করা ইত্যাদি), তখন কাস্টমাইজ করতে হয়।

সারসংক্ষেপে:

প্রশ্ন	উত্তর
User এবং Group রিলেশন কি থাকবে?	হ্যাঁ, ডিফল্ট ManyToMany থাকবে
কাস্টমাইজ না করলে কি সমস্যা হবে?	না, সব কাজ করবে ঠিকঠাক

প্রশ্ন	উত্তর
কবে কাস্টমাইজ দরকার?	বিশেষ কেসে, যেমন রিলেশন পরিবর্তন করতে চাইলে

অতিরিক্ত:

যদি তুমি `AbstractBaseUser` থেকে পুরো কাস্টম ইউজার বানাও, তবে নিজে তোমাকে `groups` ManyToMany ফিল্ড **একটু স্পষ্টভাবে** যুক্ত করতে হতে পারে। আর `AbstractUser` ইনহেরিট করলে এটা আগে থেকেই থাকে।

দ্রুতসার:

`AbstractUser` ব্যবহার করলে তোমার কোডে `groups` সম্পর্কে চিন্তা করার দরকার নাই, relation নিজে থেকেই থাকবে।

Django তে User আর Group এর Relation কেমন হয়?

---

১. ডিফল্ট ভাবে Django তে Relation

- **User** এবং **Group** এর মধ্যে থাকে **Many-to-Many** সম্পর্ক।
- অর্থাৎ, এক ইউজার অনেকগুলো গ্রুপে থাকতে পারে, আর এক গ্রুপে অনেক ইউজার থাকতে পারে।

---

২. User এবং Group এর মডেল কোড (Simplified view)

```
from django.contrib.auth.models import AbstractUser, Group

class User(AbstractUser):
    # তোমার কাস্টম ফিল্ডস...
    # e.g.
    email = models.EmailField(unique=True)

    # কিন্তু এখানে explicit M2M relation লিখতে হয় না,
    # কারণ AbstractUser থেকে inherit করলে Django internally দেয়:
    groups = models.ManyToManyField(
        Group,
        verbose_name='groups',
        blank=True,
        help_text='The groups this user belongs to.',
        related_name="user_set",
```

```
        related_query_name="user",
    )
```

---

৩. তুমি Explicitly M2M Relation লিখবে না

- কারণ Django এর `AbstractUser` বা `AbstractBaseUser` ক্লাসেই `groups` ManyToManyField already ডিফাইন করা আছে।
- তুমি কাস্টম ইউজার ক্লাসে এটা রিপিট করলে **conflict error** আসবে।

---

৪. ডাটাবেজ লেভেলে

Table Name	Description
auth_user	ইউজারের ডেটা সংরক্ষণ করে
auth_group	গ্রুপ (রোল) সংরক্ষণ করে
auth_user_groups	ইউজার এবং গ্রুপের Many-to-Many জয়েন্ট টেবিল

`auth_user_groups` টেবিল থাকে ২টা ফিল্ড দিয়ে:

- `user_id` (ForeignKey to User)
- `group_id` (ForeignKey to Group)

---

৫. তোমার কোডে কিভাবে ব্যবহার করবে?

```
user = User.objects.get(email='example@example.com')
groups = user.groups.all()  # ইউজারের সব গ্রুপ পাওয়া যাবে

# ইউজারকে নতুন গ্রুপ যোগ করা:
group = Group.objects.get(name='Seller')
user.groups.add(group)

# গ্রুপ থেকে সব ইউজার পাওয়া যাবে:
users_in_seller = group.user_set.all()
```

Summary:

Relation Type	Model Field Location	Notes
User to Group	`User.groups` (M2M)	Built-in, don't re-define
Group to User	`Group.user_set` (M2M)	Reverse accessor

যদি তোমার User মডেল এমন হয়:

```
class User(AbstractUser):
    email = models.EmailField(unique=True)
    # no need to redefine groups field
```

তাহলে তুমি `user.groups` ব্যবহার করে সরাসরি গ্রুপ ম্যানেজ করতে পারবে।

==========

1. user model update korte hobe
2. manager update korte hobe. custopm manager cara kono user model customize hobe na
3. admin.py customize korte hobe
4. multi role addisgn korle group role use kora balo role gula app.py assign kora lagbe. details up show.
5. product and category model create.
6. order orderitem cart, cartItem model create.
7. django debug toolbar added.
8. json format data load using fixtures this command(python3 manage.py loaddata fixtures/data.json) fixture folder has any data. data must be json format.
9.